Lab Assignment 10.1

Task 1

Prompt :

1. Please check the below Python code and find all syntax and logical errors in it.
2. Correct the spelling mistake in the return statement and fix any missing brackets.
3. Make sure the program runs without any errors.
4. Keep the same logic but make the code correct and clear.
5. After fixing, give the corrected code with proper explanation.

Code:

```
1
2   # Calculate average score of a student
3   def calc_average(marks):
4       total = 0
5       for m in marks:
6           total += m
7       average = total / len(marks)
8       return average   # Fixed typo
9
10  marks = [85, 90, 78, 92]
11  print("Average Score is", calc_average(marks))  # Fixed bracket
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Pytho
 Assistanting coding/2303A51100_Assignment_10.1.py"
Average Score is 86.25
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. The word average was wrongly spelled and changed to average.
2. One closing bracket was missing in the print statement.
3. The function now correctly returns the calculated average.
4. The program runs successfully without any errors.

Task 2

Prompt

1. Please rewrite the given Python code according to PEP 8 style rules.
2. Use proper spacing between parameters and operators.
3. Write the function in multiple lines instead of one single line.
4. Use meaningful formatting and indentation.
5. Return clean and properly formatted Python code.

Code:

```
13
14  def area_of_rect(length, breadth):
15      return length * breadth
16
17
18  print(area_of_rect(10, 20))
```

Output:

```
Average score is 86.25
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/rahul/Desktop/AI
 Assistanting coding/2303A51100_Assignment_10.1.py"
200
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. Parameter names were improved to be more meaningful.
2. Proper spacing was added between variables and operators.
3. Function body was written on separate lines.
4. The code now follows PEP 8 formatting rules.

Task 3

Prompt:

1. Improve the readability of the given Python code without changing its logic.
2. Replace short variable names with meaningful names.
3. Add comments to explain what the function is doing.
4. Format the code clearly with proper spacing.
5. Make the program easy to understand for beginners.

Code:

```python
# Function to calculate percentage value
def calculate_percentage(total_amount, percentage_rate):
    return total_amount * percentage_rate / 100


amount = 200
rate = 15

print(calculate_percentage(amount, rate))
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Python\Python314\python.exe
 Assistanting coding/2303A51100_Assignment_10.1.py"
30.0
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. Function name was changed to meaningful name.
2. Variables were renamed clearly.
3. A comment was added to explain the function.
4. The logic remains same but code is easier to read.

Task 4

Prompt :

1. Break the repetitive welcome printing code into a reusable function.
2. Avoid writing multiple print statements for each student.
3. Use a loop to print welcome message for all students.
4. Make the program modular and reusable.
5. Keep the output same but improve structure.

Code:

```
def welcome_students(student_list):
    for student in student_list:
        print("Welcome", student)


students = ["Alice", "Bob", "Charlie"]
welcome_students(students)
```

Output:

```
 Assistanting coding/2303A51100_Assignment_10.1.py"
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. A function was created to print welcome messages.
2. Loop is used instead of repeating print statements.
3. Code is now reusable for any number of students.
4. The program is easier to maintain and modify.

Task 5

Prompt:

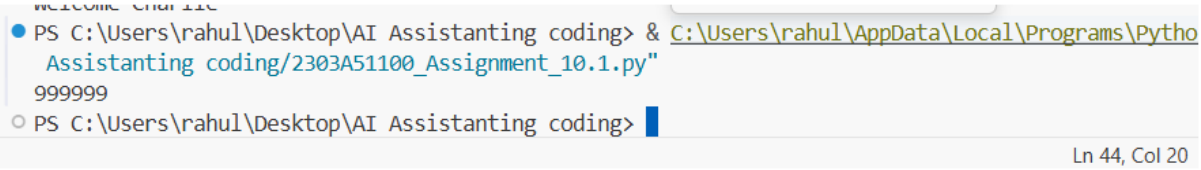
1. Improve the performance of the given Python code.
2. Remove unnecessary list creation if possible.
3. Use list comprehension to make the code faster.
4. Keep the same output but optimize execution time.
5. Return the improved and efficient version of code.

Code :

```
# Find squares of numbers
squares = [n**2 for n in range(1, 1000000)]

print(len(squares))
```

Output:



Explanation:

1. Removed extra list called nums.
2. Used list comprehension for faster execution.
3. Reduced memory usage and improved speed.
4. Output remains same as original code.

Task 6

Prompt :

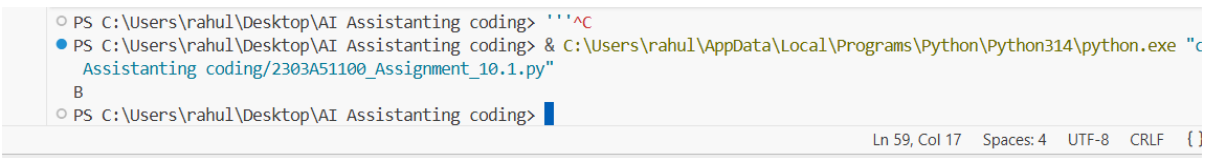
1. Simplify the nested if-else statements in the grading function.
2. Replace multiple nested blocks with elif statements.
3. Make the code cleaner and easier to read.
4. Keep the grading logic same as before.
5. Provide simplified and structured Python code.

Code:

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"


print(grade(85))
```

Output:



Explanation:

1. Replaced nested if statements with elif.
2. Code became shorter and cleaner.
3. Logic remains exactly same as before